

Edge AI Computing for Grape Leaf Disease Detection



UNIVERSITÀ DI PISA



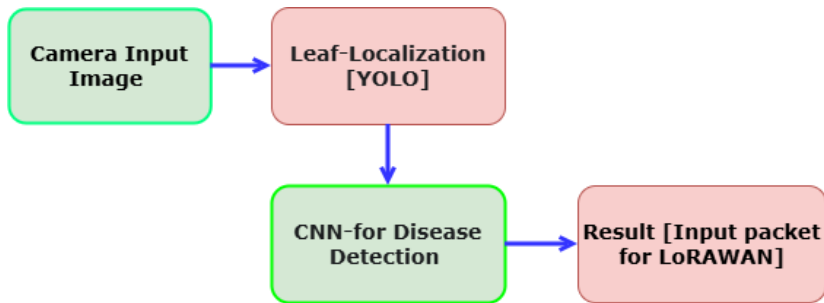
Thesis Progress...

February 8, 2026

Proposed System Architecture

- *Four-Stage Pipeline:*

- Stage 1: Input image captured using ESP32-S3 camera
- Stage 2: Localization: A lightweight model to detect leaf region (YOLO)
- Stage 3: Classification: Disease detection model (MobileNetV2)
- Stage 4: Sending only the classification result via LoRaWAN



Updates:

- 1 Previous Recap
- 2 ESP32-S3 Setup
 - ESP32-S3 WROOM Interfaces
 - MicroPython Firmware Configuration
 - Camera Configuration and Resolution selection
- 3 Data Preparation to train the leaf localization model (YOLO)
 - Data from Kaggle
 - Collected Data from the Field
 - Annotation Methodology
- 4 Next Steps
- 5 References

Previous Recap

The **MobileNetV2** architecture was successfully fine-tuned and validated on the standard Kaggle dataset. It achieved high accuracy across the four target leaf disease classes (Black Rot, Esca, Healthy, Leaf Blight).

The observed problem was:

- **Training Data:** Single, centered leaves (Uniform background)
- **Input image from camera:** Complex scenes (Multiple leaves, branches, and other backgrounds)

The **proposed solution** was to implement a **YOLO-based localization layer** to extract leaves before classification inferences.

ESP32-S3 Hardware Interface Overview

The hardware interfaces of the ESP32-S3 WROOM are distributed as follows:

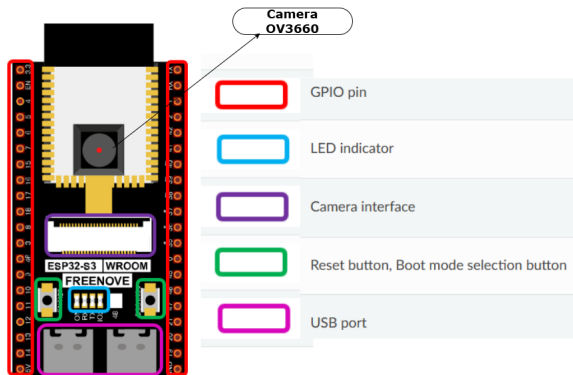


Figure: Freenove ESP32-S3 WROOM

Burning Micropython Firmware (Important)

To run Python programs on ESP32S3, we need to burn a firmware to ESP32-S3 board first.

Firmware

Releases

v1.19.1 (2022-06-18) .uf2 [.bin] [.elf] [.map] [Release notes] (latest)

v1.18 (2022-01-17) .bin [.elf] [.map] [Release notes]

Nightly builds

v1.19.1-608-gb52fe52d3 (2022-11-02) .uf2 [.bin] [.elf] [.map]

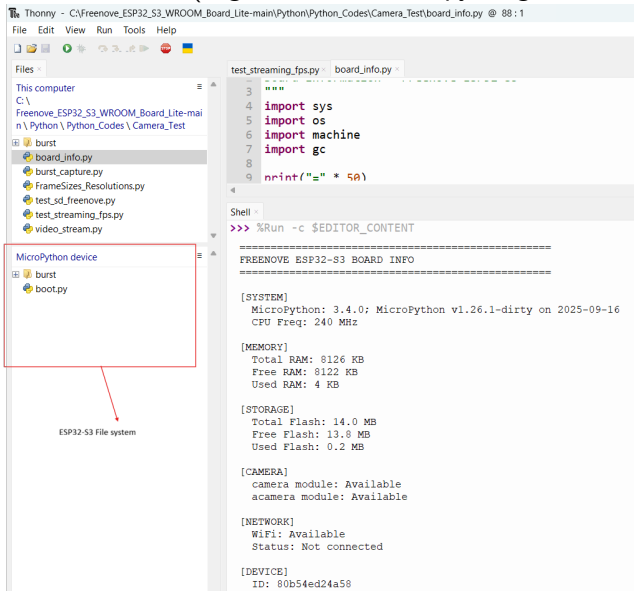
v1.19.1-607-g46bb52adf (2022-11-01) .uf2 [.bin] [.elf] [.map]

v1.19.1-604-g3ed017677 (2022-10-31) .uf2 [.bin] [.elf] [.map]

v1.19.1-601-g5b2929a0e (2022-10-31) .uf2 [.bin] [.elf] [.map]

*Before burning this firmware to ESP32S3, we need to ensure that **CH343 driver**, **Thonny**, and **Python 3** have been installed on the computer.*

Thonny IDE is utilized to write, debug, and upload Python programs directly to the ESP32-S3. (E.g. the board_info.py to get device info)



Camera Resolution and selection options

The ESP32-S3 OV3660 camera supports different levels of resolutions, ranging from 96x96 up to UXGA (1600x1200). However, as highlighted in the official documentation, there is a strict recommendation to limit the resolution to **VGA (640x480)**. Exceeding this limit may lead to memory buffer overflows and system instability.

7.1.3.2. Reference

Image resolution	Sharpness	Image resolution	Sharpness
FRAMESIZE_96x96	96x96	FRAMESIZE_HVGA	480x320
FRAMESIZE_QQVGA	160x120	FRAMESIZE_VGA	640x480
FRAMESIZE_QCIF	176x144	FRAMESIZE_SVGA	800x600
FRAMESIZE_HQVGA	240x176	FRAMESIZE_XGA	1024x768
FRAMESIZE_240x240	240x240	FRAMESIZE_HD	1280x720
FRAMESIZE_QVGA	320x240	FRAMESIZE_SXGA	1280x1024
FRAMESIZE_CIF	400x296	FRAMESIZE_UXGA	1600x1200

Table: *Official list of supported resolutions and the manufacturer's stability recommendation.*

We recommend that the resolution not exceed VGA(640x480).

Experimental Analysis on Different Resolutions

This demonstrates that while **HD** and **FHD** resolutions are technically supported, their excessive bandwidth demands (> 2000 KB/s) introduce high latency. But, **VGA (640x480)** maintains a robust frame rate (~ 27 FPS) with minimal overhead.

```
=====
Summary of Different Resolutions
=====
```

Resolution	Size	FPS	Bandwidth	Feasibility
QVGA 160x120	2.4 KB	27.8	65.6 KB/s	✓ Smooth streaming
QVGA 320x240	2.4 KB	27.8	66.2 KB/s	✓ Smooth streaming
VGA 640x480	17.6 KB	27.8	489.7 KB/s	✓ Smooth streaming
SVGA 800x600	25.0 KB	21.9	547.2 KB/s	○ Acceptable
HD 1280x720	142.0 KB	17.4	2474.0 KB/s	✗ Too slow
FHD 1920x1080	381.9 KB	17.4	6656.7 KB/s	✗ Too slow

```
=====
RECOMMENDATION
=====
```

```
Best for smooth streaming (20+ FPS): VGA (640x480)
    27.8 FPS, 17.6 KB/frame
Best quality (10+ FPS): SVGA (800x600)
    21.9 FPS, 25.0 KB/frame
```

```
>>
```

Data Preparation for Leaf Localization (YOLO)



(a) Raw Camera Input



(b) Ground Truth Annotation

- **The Requirement:** To train the YOLO model, the system requires "Ground Truth" data where every leaf instance is explicitly defined.
- **Annotation Process:**
 - Labeling specific "Leaf" regions in complex vineyard scenes.
 - Creating a custom dataset that pairs images with **spatial coordinates** (bounding boxes/polygons).

The Dataset From Kaggle

The "Grape/Grapes" dataset from Kaggle (authored by Nicolaas Regnier) contains 341 different image instances. Unlike standard datasets, which are single leaves on uniform backgrounds, this collection captures the **unstructured complexity** of a real vineyard.



Realistic Conditions:

- Include different backgrounds
- Multiple overlapping leaves per image
- Mimics the ESP32-S3 camera input

Source: Kaggle (N. Regnier)

Data Collected from the Vineyard fields

In addition to the public Kaggle dataset, I am incorporating the images collected from the vineyard fields by **Prof. Angela and her collaborators**.



Figure: Samples collected from the field (Source: Prof. Angela)

Annotation Methodology: AI-Assisted Labeling (SAM3)

To establish the ground truth for the YOLO localization model, I'm using an automated workflow using **Meta's Segment Anything Model (SAM3)** integrated in **Roboflow** platform.

Traditional Approach

- **Manual Process:** Requires drawing complex polygons point-by-point
- **Inefficient:** Extremely time-consuming
- **Inconsistent:** Prone to human error and irregular boundaries

SAM-Assisted Approach

- **Fast Workflow:** Drastically reduces annotation time per image
- **Consistency:** Ensures uniform bounding boxes with minimal manual adjustment

Examples



[click here for more examples](#)

Next Steps

With the hardware configured and the dataset prepared, the upcoming development phase focuses on training and integration.

1. Localization Model Training

- **Training:** a lightweight YOLO architecture using an annotated dataset.
- Optimizing the model for ESP32's resource constraints.

2. Embedded System Integration

- Upload both the *YOLO* (Locator) and *MobileNetV2* (Classifier) to the ESP32-S3 board.
- Develop the main MicroPython inference loop:

Capture Image → Crop ROI (YOLO) → Classify Disease (MobileNet)

References



Xu Tao, Jackson Butcher, Simone Silvestri, and Sajal K. Das (2025)
iCrop+: An Edge-boosted Crop Disease Detection System via TinyML and LoRa Communication.

In 2025 IEEE International Conference on Pervasive Computing and Communications Workshops and other Affiliated Events (PerCom Workshops), pp. 558–560.



Md Jawadul Karim, Md Omaer Faruq Goni, Md Nahiduzzaman, Mominul Ahsan, Julfikar Haider, and Marcin Kowalski (2024)

Enhancing agriculture through real-time grape leaf disease classification via an edge device with a lightweight CNN architecture and Grad-CAM.

Scientific Reports, 14(1), 16022.



Ning Zhang, Enxu Zhang, Guowei Qi, Fei Li, and Cheng Lv (2025)

Lightweight grape leaf disease recognition method based on transformer framework.

Scientific Reports, 15(1), 28974.



Grape Leaf Disease Dataset:

<https://www.kaggle.com/datasets/bmshahriaalam/grape-leaf-disease>



Thank You

Questions?